

BrokerVerse OOTB

Insurance broking platform by iorta TechNXT

Solution Architecture · Document 06 of 11

Capacity and Performance

Product	BrokerVerse OOTB
Document	06 Capacity and Performance
Version	1.1
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse OOTB Solution Architecture: 06 Capacity and Performance
Document ID	BV-SA-06
Version	1.1
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, deploy/, docker-compose.yml, .github/workflows, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue
1.1	29 September 2026	iorta TechNXT	Updated to the current system: placement journey, co-insurance, posting rules, period end and BIR tax, bank reconciliation, document numbering, commission rate matrix, single PDF engine, uploads and go-live imports, housekeeping, business time zone, broker roles, deployment on the BrokerVerse URL

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
2 Built-in capacity controls.....	5
2.1 Request and upload limits.....	5
2.2 Performance mechanisms.....	5
2.3 Known performance risks in the code.....	6
3 Indicative load test.....	7
3.1 Method.....	7
3.2 Results.....	7
3.3 Interpretation.....	8
4 Data growth and storage.....	9
4.1 Current relative weight (test data).....	9
4.2 Growth estimate.....	9
5 Sizing recommendation.....	11
6 Performance test plan for UAT.....	12
6.1 Objectives.....	12
6.2 Response-time targets.....	12
6.3 Approach.....	12
7 Related documents.....	14
8 Glossary.....	15

1 Introduction

1.1 Purpose and scope

This document describes the capacity and performance characteristics of BrokerVerse OOTB: the limits and performance mechanisms built into the code, the results of an indicative load test on the build host, the expected data growth, a sizing recommendation for AWS and the performance test plan for UAT.

Recommended / to be confirmed by the business and DevOps: The business volumes used for sizing (users, policies per year, documents per policy) are assumptions. Replace them with the business forecast before the infrastructure is ordered, and confirm the sizing with the UAT performance test in chapter 6.

2 Built-in capacity controls

2.1 Request and upload limits

Control	Value (default)	Where
JSON request body	2 MB	JSON_BODY_LIMIT, <code>express.json</code>
Uploaded document or photo	10 MB per file, 10 files per request	UPLOAD_MAX_MB, UPLOAD_MAX_FILES; held in memory while checked (<code>lib/uploadLimits.js</code>)
Spreadsheet or CSV import, statement file	10 MB, one file	IMPORT_MAX_MB
Workbook decompression	50 MB per workbook part	IMPORT_MAX_INFLATED_MB (protection against compressed "zip bombs")
Import rows	20,000 rows per file; bulk upload screens 1,000	IMPORT_MAX_ROWS, <code>limits.bulk_upload_max_rows</code>
Upload settings	images 2 MB, bulk files 10 MB	<code>uploads.image_max_bytes</code> , <code>uploads.bulk_max_bytes</code>
Web container request body	25 MB	<code>nginx.conf client_max_body_size</code> (Docker Compose option)
List page size	default 10, maximum 500 per page	<code>lib/respond.js paging()</code>
Report rows	50,000 per generated file; on-screen page 50; default date range 365 days	<code>reports.max_rows</code> , <code>reports.default_page_size</code> , <code>reports.default_range_days</code>
Global API rate limit	600 requests per 60 s per signed-in user (or per IP) per API process	<code>security.api_rate_limit</code>
Sign-in rate limit	10 attempts per 300 s per IP and per username per API process	<code>security.login_rate_limit</code>
Database connections	10 per API process, plus one dedicated connection per running job	<code>db/pool.js</code> (max: 10)
E-mail throughput	50 messages per outbox run, every 5 minutes (600 per hour)	<code>lib/mailler.js</code> , job email-outbox
Bank matching	Date window 5 days; one-to-many and many-to-one groups of up to 6 lines	<code>bank_reconciliation.date_window_days</code> , <code>bank_reconciliation.group_max_lines</code>

2.2 Performance mechanisms

- **Server-side paging** on lists (`LIMIT` and `OFFSET` with a total count) and on report runs; screens request one page at a time.

- **Indexes** on business numbers, status and date columns, on the foreign keys of every growing table (migration 0146), partial indexes for hot subsets (open direct-bill items, live refresh tokens, active entry matches, due recurring journals) and a GIN index on generic master data (document 03).
- **Settings cache** in memory per process (`lib/settings.js`): a key is read from the database once and the cache checks the table version at most every 5 seconds.
- **Work outside the request**: e-mail through the outbox, renewal notices through `job_queue`, scheduled reports, ageing and the period-end jobs through the scheduler.
- **Housekeeping**: the daily `housekeeping` job keeps the operational tables (job runs, outbox, sign-in history, tokens, notifications, queue) at a bounded size (document 10).
- **Static assets** served by CloudFront with long caching of hashed files and `index.html` not cached.
- **Stateless API**: capacity grows by adding API instances behind the load balancer (document 07).

2.3 Known performance risks in the code

Risk	Detail and recommendation
CPU-bound file generation	PDF, XLSX and CSV files (up to 50,000 report rows), bulk PDF prints and spreadsheet imports run in the Node.js event loop of the API process. While a large file is written, other requests on the same instance wait. Recommended: keep <code>reports.max_rows</code> as is, run large scheduled reports on an instance that is not in the web target group, or move file generation to a worker later.
Uploads in memory	Up to 10 files of 10 MB per request are held in memory. Size the container memory for concurrent uploads (at least 1 GB of head-room per instance; 2 GB recommended).
Bank matching on large statements	Automatic matching compares each bank line with the open book lines of the account within the date window; the one-to-many rules search combinations of up to <code>group_max_lines</code> lines. Very large statements (thousands of lines) are best imported per week.
OFFSET paging	Deep pages of very large lists cost more with <code>OFFSET</code> ; acceptable for the page sizes used, to be reviewed if a list passes about 100,000 rows.
Per-process pool	With N instances the database sees up to 10 x N connections plus the job lock connections; keep within the <code>max_connections</code> of the database class.

3 Indicative load test

3.1 Method

The harness `docs/architecture/tools/loadtest.mjs` starts an in-process instance of the API (the same `createApp()` as production) on a free local port, so no running system is affected. It signs an access token for the administrator the way the backend tests do and drives read-only requests at a fixed concurrency for 15 seconds per endpoint, once with 10 and once with 50 concurrent clients. The per-process rate limiter is reset every 100 ms by the test hook `resetRateLimits()` so it does not cap the measurement.

Condition	Value
Date	29 September 2026, on the code of this issue
Host	Build host: 4 vCPU, 16 GB RAM, Linux; PostgreSQL 16.13 on the same host
API	Node.js 22, one process (one event loop), pool of 10 connections, <code>LOG_LEVEL=warn</code>
Data	The database of documents 02 and 03 (28 policies, 16 leads, 15 claims): test data , far smaller than production
Client	Same host (no network latency, no TLS, no load balancer)

Note: The results are **indicative only**: one small host, client and database on the same machine, a small data set, no TLS and no network. They show the relative cost of the endpoints and that a single API process is bound by its event loop. They are not the capacity of the production environment.

3.2 Results

Clients: concurrent clients; Req/s: completed requests per second; latencies in milliseconds measured at the client; Non-2xx: responses other than 2xx.

Endpoint	Clients	Requests	Req/s	p50 ms	p95 ms	p99 ms	Max ms	Non-2xx
GET /health (readiness, DB ping)	10	31,540	2,103	4	8.3	14.7	151.5	3
GET /health/live (liveness)	10	57,829	3,855	2.2	4.5	6.4	24.9	0
GET /policies?page=1&perPage=10	10	10,274	685	13.5	23.4	33.5	148.1	0
GET /leads?page=1&perPage=10	10	15,181	1,012	9.7	12.1	14.9	34.4	0
GET /claims?page=1&perPage=10	10	8,636	575	16.4	24.7	37.3	79.3	0
GET /dashboard/executive	10	6,069	404	23.1	34.7	46	216.3	0
GET /notifications/unread-count	10	18,361	1,224	7.7	11.5	18.9	124.7	0
GET /search?q=santos	10	12,740	849	11.2	16.6	22.4	49.7	0
POST /reports/production-register/run (on-screen preview)	10	8,076	538	17.7	25.2	37.1	81.9	0
GET /health (readiness, DB ping)	50	43,950	2,929	15.6	26.1	36.5	385.6	3
GET /health/live (liveness)	50	40,230	2,680	18	25.4	34.5	229.5	0
GET /policies?page=1&perPage=10	50	9,759	648	73.3	111.3	132	183.5	0
GET /leads?page=1&perPage=10	50	15,144	1,008	47	60.9	80.3	107.2	0
GET /claims?page=1&perPage=10	50	9,269	616	76.8	109.6	132	157.2	0
GET /dashboard/executive	50	5,861	388	124	174.2	231.2	249.3	0
GET /notifications/unread-count	50	21,491	1,431	31.8	53.7	72.6	107.5	0
GET /search?q=santos	50	12,443	828	54.7	82.9	97	128.3	0

Endpoint	Clients	Requests	Req/s	p50 ms	p95 ms	p99 ms	Max ms	Non-2xx
POST /reports/production-register/run (on-screen preview)	50	8,266	550	87	119.3	147.7	171.3	0

3.3 Interpretation

- At 10 concurrent clients one API process served about 400 to 1,000 requests per second on the list, dashboard, search and report-preview endpoints, with 95th-percentile latencies between 12 and 35 ms, and about 1,200 per second on the notification unread count that every signed-in browser polls.
- At 50 concurrent clients the throughput stayed about the same while latency grew about four to six times (95th percentile 54 to 174 ms). One Node.js process is saturated at a few hundred requests per second of this mix, so capacity grows by adding instances, not by concurrency.
- The executive dashboard, the policy and claim lists and the report preview are the most expensive of the measured reads; the unread-count poll is cheap but frequent (one request per signed-in user every 30 seconds).
- 3 of about 31,500 readiness checks at 10 clients and 3 of about 43,950 at 50 clients answered 503; the cause was not isolated in this test and the liveness check had none. A load balancer marks a target unhealthy only after several failures in a row, so single 503s do not remove an instance. Re-check in UAT with the production health-check settings.
- With production volumes (hundreds of thousands of rows) the database share of each request grows; the UAT test must use a volume-loaded database.

4 Data growth and storage

4.1 Current relative weight (test data)

Table	Rows (test data)	Total size KB	Average row bytes	Retention class
app_settings	351	144	199	Reference
master_records	319	440	394	Reference
journal_lines	210	152	185	Financial
role_permissions	163	24	32	Reference
gl_accounts	129	88	173	Reference
posting_rule_lines	100	80	109	Reference
journal_vouchers	86	264	334	Financial
schema_migrations	58	32	61	Reference
permissions	56	48	78	Reference
master_types	53	104	915	Reference
document_numbering	52	96	182	Reference
cities	49	64	135	Reference
claim_field_changes	48	48	122	Log / audit
claim_history	48	48	88	Log / audit
product_components	48	104	406	Reference

4.2 Growth estimate

The estimate multiplies the average row sizes measured in the test database by assumed business volumes. Index and page overhead roughly doubles the heap size.

Assumption (Recommended / to be confirmed by the business)	Value	Basis
Named users / concurrent users at peak	200 / 80	to be confirmed
Policies issued per year (new and renewal)	50,000	to be confirmed
Endorsements, claims per year	10,000 / 3,000	to be confirmed
Uploaded documents and photos per policy	4 files, 0.5 MB average	to be confirmed
Generated files (reports, statements, schedules)	2 GB per year, 90-day retention for reports	setting <code>reports.retention_days</code>

Item	Estimate per year	Calculation
Transaction rows per policy (quotation 0.8 KB, policy 0.5 KB, bill, booking and receipt journals with lines, co-insurance participants, commission, receipt and application, collection item, remittance line)	about 5 KB heap	measured average row sizes
Audit, notification, e-mail and document rows per policy (about 10 audit rows, 2 notifications, 2 e-mails, 4 document rows)	about 6 KB heap	estimate

Item	Estimate per year	Calculation
Database growth from policies, including index overhead	about 1.1 GB	$50,000 \times 11 \text{ KB} \times 2$
Endorsements, claims, renewals, placement, reinsurance, bank statements and period-end journals	about 0.5 GB	estimate
Scheduler history (<code>job_runs</code> : 1,736 rows a day, kept 90 days by housekeeping)	under 0.1 GB, steady	from the job schedule
Sign-in history, refresh tokens (200 users, 4 sessions a day; kept 365 and 30 days)	under 0.1 GB, steady	housekeeping periods
Database total	about 2 GB per year (plus WAL and backups)	
File store (uploads)	about 100 GB per year	$50,000 \times 4 \times 0.5 \text{ MB}$, rounded up

The database stays small; the file store drives storage growth and backup volume.

5 Sizing recommendation

Recommended / to be confirmed by the business and DevOps: Starting sizes for production in AWS ap-southeast-1, based on the assumptions above. Confirm them with the UAT performance test and adjust after the first months of operation.

Component	Recommended start	Scaling rule
API containers	2 instances of 1 vCPU and 2 GB; the scheduler may run on both (advisory lock) or on one (SCHEDULER_ENABLED)	Scale on average CPU 60%; maximum 6 instances; about one instance per 250 requests per second of peak load
Load balancer	HTTPS, idle timeout 120 s (large report downloads and imports), health check <code>/api/health</code> every 15 s, 3 failures to mark unhealthy	Managed
Database	Amazon RDS for PostgreSQL 16, <code>db.m7g.large</code> (2 vCPU, 8 GB) Multi-AZ; <code>db.t4g.medium</code> for UAT	Scale the class up on sustained CPU above 60% or connections above 70% of <code>max_connections</code>
Database storage	gp3, 100 GB with storage autoscaling to 500 GB	Alarm at 80% used
File store	Amazon EFS, General Purpose, elastic throughput, lifecycle to Infrequent Access after 90 days	Grows automatically; alarm on monthly growth
Static front end	S3 and CloudFront	Managed
Connections	10 per API instance, plus one per running job	6 instances = about 70 connections, well within the RDS limit

6 Performance test plan for UAT

6.1 Objectives

1. Confirm that the production sizing meets the response-time targets below at the expected peak load plus 50% head-room.
2. Find the saturation point of one API instance and the scaling with two and four instances.
3. Confirm that scheduled jobs, report generation, bulk prints, imports and bank matching do not push interactive response times past the targets.
4. Record a baseline for regression tests of later releases.

6.2 Response-time targets

Transaction class (Recommended / to be confirmed by the business)	95th percentile	Maximum error rate
Navigation, lists, details, search, notification count	1.0 s	0.1%
Save or submit (lead, quotation, broker slip, receipt, voucher, claim)	2.0 s	0.1%
Quotation pricing, policy issue (with billing, co-insurance and journals)	3.0 s	0.1%
Dashboards, on-screen reports (one page)	3.0 s	0.5%
Report file up to 50,000 rows, import of 1,000 rows, month-end close run	60 s	1%

6.3 Approach

Step	Content
Environment	UAT on the production sizing (chapter 5), with the same database, file store, load balancer and CloudFront set-up as production; <code>LOG_LEVEL=info</code>
Data	A volume-loaded copy: at least one year of forecast volume (for example 50,000 policies with bills, receipts, journals and commissions; 3,000 claims; 200 users), generated with a data script on UAT; never production data without masking
Tool	k6, JMeter or Artillery scripts per role, using the API (tokens from <code>POST /api/auth/login</code> for test users; one user per virtual user to stay under the per-user rate limit, or <code>security.api_rate_limit</code> raised for the test)
Workload mix	Sales & Marketing (search, leads, quotations), Processing Team (broker slips, placement, policy issue), Operations (policy views, endorsements), Accounting (receipts, journal vouchers, remittance, bank matching), Claims, dashboards and reports, the 30-second notification poll per user
Test types	Baseline (1 user), load (expected peak of 80 concurrent users for 1 hour), stress (increase to failure), soak (peak load for 8 hours with the scheduler running), spike (5 x peak for 5 minutes)
Measurements	Client latencies and errors; load balancer target response time and 5xx; container CPU and memory; database CPU, connections, IOPS, top SQL (Performance Insights); <code>job_runs</code> durations; event-loop delay
Exit criteria	Targets met at peak plus 50%; no memory growth in the soak test; no errors other than intended 429s; database CPU below 60% at peak
Deliverable	Test report with the measured capacity per instance, the confirmed sizing and the slow SQL statements with their index or query fixes

7 Related documents

This document is one of eleven that together form the BrokerVerse OOTB Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance (this document)	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

8 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (pg_try_advisory_lock, pg_advisory_lock) used so a job runs on one API instance at a time and only one instance applies migrations.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
Broker slip	Request for quotation (BS-) that the broker sends to several insurers for one client risk; the answers are recorded as insurer offers.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
Co-insurance	Placement of one risk with several insurers, one of them the lead, each taking a share; premium, commission and claims are split by share.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
GIN	Generalised inverted index (PostgreSQL), used for JSONB searches.
Housekeeping	Daily job that deletes operational rows (job runs, sent e-mails, sign-in history, expired tokens, read notifications) after the retention days in System Settings.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Month-end close	Run (MEC-) that posts the accruals, recurring journals and valuations of a period, checks the checklist and, once approved, closes the period.
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
nginx	Web server used in the web container to serve the SPA and forward /api to the API container.
p50 / p95 / p99	Latency percentiles: the time within which 50 %, 95 % or 99 % of requests completed.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
TLS	Transport Layer Security (HTTPS).
UAT	User acceptance testing environment.
WAL	Write-ahead log: PostgreSQL change log used for replication and point-in-time recovery.